

First Come, First Served (FCFS)

FCFS serves processes in arrival order. Once a process starts, it keeps the CPU until it finishes.

1. Model

The visualizer uses one CPU and whole ticks. Processes need only CPU service, with no I/O blocking. **Service time** is the total CPU work needed to finish. Each execution tick reduces remaining service by one. Switching processes takes no time.

A future process has not arrived yet. A ready process has arrived and is waiting for the CPU.

Queue entries read from left to right; [] means empty. Each timeline cell is one tick, and a dash (-) means the CPU is idle. Examples under different rules are separate scenarios.

2. Scheduling rules

Rule 1. Join the back of the queue on arrival

Append new processes to the ready-queue tail. Simultaneous arrivals enter in input order. Existing waiting processes keep their places.

Example. B is waiting. C and A arrive together, listed in that order.

```
Before arrivals:  head -> [B]          <- tail
New arrivals:           [C] [A]
After arrivals:   head -> [B] [C] [A] <- tail
```

Rule 2. Run the process at the front

When the CPU is free, remove the queue head and run it. Service time and process ID do not affect selection.

Example. B needs five ticks; C needs one.

	Before dispatch	After dispatch
CPU	free	B
Ready queue	[B] [C]	[C]

B is selected even though C is shorter.

Rule 3. Let the running process finish

FCFS is **non-preemptive**. New arrivals wait; no quantum forces the current process to give up the CPU.

Example. A arrives at 0 with service 5. B arrives at 1 with service 1.

```

Time 0   1   2   3   4   5   6
      +---+---+---+---+---+---+
CPU   | A | A | A | A | A | B |
      +---+---+---+---+---+---+

```

t=1: B arrives -> waits while A continues.

t=5: A finishes -> B starts.

Rule 4. Complete at zero remaining service

Remove the finished process before selecting another. Arrivals at this boundary join behind existing waiters. The next process may start immediately.

Example. A finishes at time 5, B is waiting, and C arrives at 5.

```

At t=5:
1. Finish A      CPU: free   Ready: [B]
2. Admit C       CPU: free   Ready: [B] [C]
3. Dispatch B    CPU: B      Ready: [C]

```

Rule 5. Idle only when nothing is ready

If both CPU and ready queue are empty, wait for the next arrival. An empty queue does not stop a running process. Stop when every process has finished.

Example. A arrives at 0 with service 2; B arrives at 5 with service 1.

```

Time 0   1   2   3   4   5   6
      +---+---+---+---+---+---+
CPU   | A | A | - | - | - | B |
      +---+---+---+---+---+---+

```

A finishes at 2. B is still a future process until 5.

The three idle cells are the interval from 2 to 5.

3. Events at a tick boundary

First account for execution during the preceding tick. Then:

1. Remove the running process if it has finished.
2. Add all arrivals at this time to the queue tail, in input order.
3. If the CPU is free, dispatch the queue head.

4. Worked example

Input order is A, B, C. All times are in ticks.

<i>Process</i>	<i>Arrival</i>	<i>Service</i>
A	0	5
B	1	2
C	1	1

```

Time 0   1   2   3   4   5   6   7   8
      +---+---+---+---+---+---+---+
CPU   | A | A | A | A | A | B | B | C |
      +---+---+---+---+---+---+---+

```

t=0: A starts.

t=1: B and C arrive; ready queue is [B] [C].

t=5: A finishes; B runs first because it entered first.

t=7: B finishes; C starts.

t=8: C finishes; all processes are done.